CAB301 Assignment 1

Analysis Answer Sheet

Student Name: Huynh Anh Quan Dang

Student Number: n12051632

Unit: CAB301

Document Type: Assignment Submission

Instructions

Answer all questions clearly and concisely.

For Questions 1–4:

- state a suitable **basic operation**
- count how many times it is performed in the **worst case**
- give the resulting **Big-O time complexity**, using the variables defined below

Any correct worst-case count is acceptable if the reasoning is clear and consistent.

Notation

- b = number of books currently stored
- e = number of recorded borrow events
- m = number of recorded borrow events for a given Dewey number
- t = capacity of the `DeweyCountTable`

1. Starter `ContainsBook`

The starter implementation performs a linear scan over the catalogue.

(a) Basic operation

The parameter that characterises the problem size is b, the number of books currently stored.
The basic operation is the comparison:

```
mBooks[i].DeweyNumber == deweyNumber
```

This is the key operation repeated during the scan, because for each book the method checks whether that book's Dewey number matches the target Dewey number.

(b) Worst-case count

In the worst scenario, the target Dewey number is **not present** in the catalogue, or it is stored in the **last position**.
In that case, the method must compare against every stored book.
Thus, the basic operation will be performed b **times** in the worst case.

(c) Big-O complexity

Since the basic operation is performed at most b **times**, the worst-case time complexity of the algorithm is:

$$\mathbf{O}(b)$$

2. Completed `SortCatalogue`

(a) Briefly justify your choice of algorithm

I used **merge sort** because it is a reliable and efficient sorting algorithm. It works well for arrays and guarantees good performance even in the worst case.

(b) Basic operation

The parameter that characterises the problem size is b, the number of books in the catalogue, and the basic operation is **comparing two Dewey numbers** when deciding which item should come first during the merge step.

(c) Worst-case count

Now we calculate the total number of times the basic operation will be performed.
In merge sort, the catalogue is repeatedly divided into smaller subarrays until each subarray has only one book. Then these subarrays are merged back together in sorted order.
When two sorted subarrays are merged, the algorithm repeatedly compares the front elements of the two subarrays. These comparisons are the basic operations.
The array is repeatedly split into halves:

- after 1 split, there are 2 parts
- after 2 splits, there are 4 parts
- after 3 splits, there are 8 parts

This continues until the subarrays have size 1. The number of times we can keep dividing b by 2 until we reach 1 is approximately $\log b$.
At each level of this divide-and-merge process, all b books are involved in the merging work, so the total number of comparisons at one level is proportional to b.
Since there are $\log b$ levels, and each level performs about b basic operations, the total number of basic operations in the worst case is about:

$$b \log b$$

Thus, the worst-case count of the basic operation is **about** $b \log b$.

(d) Big-O complexity

Therefore, the worst-case time complexity of the algorithm is:

$$\mathbf{O}(b \log b)$$

3. Completed `GetBookIndex`

Assume the catalogue is currently marked sorted.

(a) Briefly justify your choice of algorithm

I used **binary search** because the catalogue is already sorted. Binary search allows us to find a book much faster than scanning the whole list.

(b) Basic operation

The parameter that characterises the problem size is b, the number of books in the catalogue, and the basic operation is **comparing the target Dewey number with the middle element**.

(c) Worst-case count

Now we calculate the total number of times the basic operation will be performed.
In binary search, instead of checking every element, we repeatedly divide the search space into half:

- after 1 comparison, the remaining search space is $b/2$
- after 2 comparisons, it becomes $b/4$
- after 3 comparisons, it becomes $b/8$

This process continues until the search space becomes 1.
The number of times we can divide b by 2 until we reach 1 is approximately $\log b$.
At each step, only one comparison is performed, which is comparing the target with the middle element.
Thus, the total number of basic operations in the worst case is **about** $\log b$.

(d) Big-O complexity

Therefore, the worst-case time complexity of the algorithm is:

$$\mathbf{O}(\log b)$$

4. Starter `FindBorrowLimitViolator`

Ignore the initial `ContainsBook` check.

The starter implementation:

- scans all e events to collect the matching ones
- then repeatedly rescans the m matching events to count repeated student IDs

(a) Basic operation

The parameters that characterise the problem size are e, the number of recorded borrow events, and m, the number of matching borrow events for the given Dewey number.
The basic operation is **comparing borrowers' studentID between matching borrow events**.

(b) Worst-case count (in terms of e and m)

Now we calculate the total number of times the basic operation will be performed.
First, the algorithm scans all e borrow events to collect the ones that match the given Dewey number.
Then, after collecting the m matching events, the algorithm repeatedly rescans them to count repeated student IDs.
To calculate the total number of comparisons among the matching events, we break the problem into cases:

- when checking the first matching event, it may be compared with all m matching events
- when checking the second matching event, it may again be compared with all m matching events
- this continues for all m matching events

Thus, the total number of student ID comparisons in the worst case is proportional to:

$$m \times m = m^2$$

Including the initial scan of all borrow events, the total worst-case count is:

$$e + m^2$$

(c) Big-O complexity

Therefore, the worst-case time complexity of the algorithm is:

$$\mathbf{O}(e + m^2)$$

5. Improved `FindBorrowLimitViolator`

(a) Explain how your method achieves $\mathbf{O}(e + m \log m)$ or better

The parameters that characterise the problem size are e, the number of recorded borrow events, and m, the number of matching borrow events for the given Dewey number.

My method first scans all e borrow events once to collect the matching events for the given Dewey number. This part takes $\mathbf{O}(e)$ time.

After that, it processes only the m matching events. Instead of repeatedly rescanning them, it sorts the matching student IDs. I used a sorting algorithm with worst-case time $\mathbf{O}(m \log m)$.

To understand where $m \log m$ comes from:

- m comes from the number of matching events that need to be sorted
- $\log m$ comes from repeatedly dividing the list during the sorting process
- at each level, about m items are involved
- since there are about $\log m$ levels, the total work is $m \log m$

After sorting, the method makes one more linear scan through the sorted student IDs to check whether any student ID appears 3 or more times. This takes $\mathbf{O}(m)$ time.

So the total running time is:

$$\mathbf{O}(e) + \mathbf{O}(m \log m) + \mathbf{O}(m)$$

Since $m \log m$ grows faster than m, this becomes:

$$\mathbf{O}(e + m \log m)$$

(b) Worst-case Big-O complexity in terms of e and m

Therefore, the worst-case time complexity of the method is:

$$\mathbf{O}(e + m \log m)$$

6. `MostBorrowedSubject` with the Auxiliary Structure

The completed system maintains a `DeweyCountTable` incrementally.

Assume linear probing takes expected $\mathbf{O}(1)$ time per table operation.

(a) Time complexity of updating the table during one successful `RecordBorrowEvent`

The parameter that characterises the problem size here is the table operation itself.
During one successful `RecordBorrowEvent`, the system updates the `DeweyCountTable` by incrementing the count for one Dewey number.
Since the question states that linear probing takes expected $\mathbf{O}(1)$ time per table operation, the time complexity of this update is $\mathbf{O}(1)$ **expected time**.

(b) Worst-case time complexity of `MostBorrowedSubject()` in terms of t

To find the most borrowed subject, the method scans the table to find the key with the largest count.
In the worst case, it may need to check all t positions in the table.
Thus, the worst-case count of the basic work is proportional to t, so the worst-case time complexity is:

$$\mathbf{O}(t)$$

(c) Why is maintaining the table incrementally more efficient than rescanning all borrow events whenever `MostBorrowedSubject()` is called?

This means that when `MostBorrowedSubject()` is called, the system does not need to scan all borrow events again and recompute the counts from the beginning.
Instead, it can use the counts that have already been stored in the auxiliary table, which avoids repeating the same work many times.

(d) If the system did not maintain `DeweyCountTable` incrementally and instead recomputed counts from all borrow events each time `MostBorrowedSubject()` is called, what would be the worst-case time complexity of one call in terms of e?

The parameter that characterises the problem size is e, the number of recorded borrow events.
If the system did not maintain the table incrementally, then each time `MostBorrowedSubject()` is called, it would need to scan all e borrow events to recompute the counts.
In the worst case, this requires one full scan of the borrow events, so the worst-case time complexity of one call would be:

$$\mathbf{O}(e)$$

End of Report